TINT: flujo del vídeo y reparto de responsabilidades

Guía sencilla para entender quién hace qué desde que se sube un vídeo hasta que se ve en una sala. Estado revisado el 11 de octubre de 2026 contra la rama *develop*.

1. La idea en pocas palabras

TINT es una sala para ver vídeos en grupo, todos al mismo tiempo. Para eso un vídeo pasa por tres etapas: el anfitrión lo **sube**, un trabajador lo **prepara** (lo revisa, lo deja en un formato estándar y calcula los colores del efecto Ambilight) y, cuando está listo, se **reproduce** sincronizado en la sala. El objetivo del prototipo son películas de 2 a 3 horas.

2. Recorrido del vídeo, paso a paso

#	Qué pasa	Quién se encarga
1	El anfitrión inicia sesión (Google, mediante Firebase) y elige un vídeo mp4.	Página web + servicio de identidad
2	Se reserva el vídeo y se entrega una dirección temporal para subirlo directamente al almacén de archivos (MinIO). El archivo no pasa por la API.	Servicio de identidad
3	El navegador sube el archivo. Al terminar, confirma la subida y el vídeo queda en estado **processing**. Se avisa al worker por una cola (Redis).	Página web + servicio de identidad
4	El worker revisa que el archivo no esté dañado y obtiene su duración exacta.	Media worker
5	El worker deja el vídeo en un formato estándar (MP4 H.264 + AAC) para que se vea igual en todas partes. **Este paso aún falta por construir.**	Media worker
6	El worker calcula los colores de los bordes del vídeo (un dato por segundo) para el efecto Ambilight.	Media worker
7	El worker guarda el vídeo final y el archivo de colores en el almacén, y marca el vídeo como **ready** (o **failed** si algo salió mal).	Media worker
8	El anfitrión asigna el vídeo a su sala. Quien entra recibe una dirección temporal para descargarlo, solo si es el dueño o un invitado de esa sala.	Servicio de identidad
9	Todos ven el vídeo al mismo tiempo; el anfitrión manda pausa, avance y cambios.	Sync-core (sincronía) + página web

3. Estados de un vídeo

Reservado y esperando la subida (**pending**) > en preparación (**processing**) > terminado (**ready**) o con error (**failed**). El primer cambio lo hace el servicio de identidad; los dos últimos los hace el media worker. Mientras un vídeo no esté **ready**, la web no deja usarlo en una sala y muestra su estado.

4. Quién se encarga de cada cosa

Dominio	Responsabilidad	Dónde está
Página web	Pantallas: login, salas, subida del vídeo, reproductor y chat.	apps/web

Dominio	Responsabilidad	Dónde está
Servicio de identidad	Usuarios, salas, invitaciones y permisos. Valida el login de Firebase, reserva y confirma las subidas, firma las direcciones temporales y es dueño de la tabla *videos* en la base de datos.	services/identity-api
Media worker	Prepara el vídeo: revisión, formato estándar y colores Ambilight. Publica el resultado y actualiza el estado.	services/media-worker
Sync-core	Mantiene a todos en el mismo segundo del vídeo (reloj maestro).	services/sync-core
Social-core	Chat y reacciones en vivo.	services/social-core
Analytics	Métricas del uso, sin datos personales. Definirá los metadatos del vídeo.	services/analytics
Infraestructura	Docker compose (base de datos, Redis, MinIO), pruebas automáticas y despliegue.	docker-compose.yml, .github

5. TODO: qué falta y quién lo hace

Lo que se necesita para que el prototipo funcione con películas largas. Cada tarea va en un cambio (PR) del dominio indicado.

Prioridad	Tarea	Dominio
Alta	Convertir el vídeo al formato estándar MP4 H.264 + AAC (archivo final *source.mp4*) antes de calcular el Ambilight.	Media worker
Alta	Probar con películas reales de 2 a 3 h: cuánto tarda la conversión y cuánta memoria usa.	Media worker
Alta	Revisar que una conversión larga no haga que la cola entregue el mismo vídeo a otro worker, y que el tiempo máximo de FFmpeg alcance para una película.	Media worker
Alta	Decidir y ajustar los límites de subida (hoy 2 GiB, que se queda corto para películas) con los datos de las pruebas.	Servicio de identidad
Media	Confirmar si el proxy o la puerta de entrada limita el tamaño de las peticiones.	Infraestructura
Media	Acciones del dueño sobre sus vídeos: listar, borrar (también del almacén) y reintentar los que fallaron.	Servicio de identidad + web
Media	Asignar formalmente quién mantiene el esquema de la tabla *videos* (hoy es del servicio de identidad).	Servicio de identidad
Media	Borrar la carpeta *videos/* de ejemplos del media worker (los archivos llegan por la subida) tras comprobar que ninguna prueba la usa.	Media worker
Baja	Definir qué metadatos guardar (códecs, resolución) junto con analytics.	Analytics + identidad

6. Para el siguiente prototipo

Tarea	Dominio
Emitir el vídeo por trozos (HLS, como YouTube) en lugar de un solo archivo.	Media worker + sync-core + web

Tarea	Dominio
Aceptar otros formatos de entrada además de mp4.	Media worker
Quitar Redis del flujo de vídeo y que el worker tenga su propia API y proceso en segundo plano.	Media worker + identidad
Biblioteca de vídeos reutilizable en varias salas (hoy cada vídeo pertenece a una sola sala).	Servicio de identidad + web

7. Decisiones ya tomadas

• Se mantiene el flujo actual de *develop*: el navegador sube directo al almacén y el worker procesa desde una cola. No se rediseña en este prototipo.

• El estado mientras se prepara el vídeo es **processing**.

• El vídeo nunca se publica en el almacén sin haber sido revisado y preparado.

• La autenticación (Firebase/Google) la valida el servicio de identidad; el worker no autentica.

• Formato estándar: MP4 H.264 + AAC. Por ahora solo se aceptan vídeos mp4.